

General Diagram Specification Guide

FraudWatch AI Engineering Specifications • Generated Document

Software Engineering Diagram Feature & Specification Guide

This document provides a complete specification of features, elements, notations, and requirements for creating Data Flow Diagrams (DFD) (including multi-level hierarchy), Use Case Diagrams, Entity-Relationship (ER) Diagrams, and Activity Diagrams. It also details common shared diagram-editor features.

1. Core / Shared Diagramming Features

Every diagram tool requires a common set of foundational canvas and manipulation capabilities. Even if diagram types differ, these core features must be present across all diagram models:

1.1 Canvas & Workspace Management

- Infinite / Flexible Grid Canvas: Support for pan, smooth zoom (10% to 500%), snap-to-grid, and alignment guides.
- Multi-Page & Tab Support: Ability to maintain multiple pages or level sub-sheets within a single project file.
- Dark / Light Theme & Styling: Custom colors, line weights, font selections, fills, shadows, and opacity controls for all shapes.
- Auto-Layout Algorithms: Automatic tree, orthogonal, force-directed, and layered grid layouts for automatic alignment of shapes.

1.2 Interactive Element Operations

- Drag-and-Drop Stencils: Palette of domain-specific shapes categorized by diagram type (DFD, Use Case, ER, Activity).
- Smart Connector Routing: Dynamic connection points with automatic obstacle avoidance, orthogonal (right-angle), straight, and curved line styles.
- Text Editing & Labeling: In-line text editing for nodes, relationship labels, cardinality indicators, and process names.
- Grouping & Containerization: Group multiple shapes into sub-containers, frames, or sub-graphs.
- Undo / Redo & Version History: Full action stack for non-destructive editing and historical snapshots.

1.3 Export, Import & Collaboration

- Export Formats: Vector (SVG, PDF) and Raster (PNG, JPEG), plus structured code/data (JSON, XML, Mermaid syntax).
- Import Capabilities: Import from standard formats (Mermaid, PlantUML, GraphML, SQL DDL scripts).
- Validation Engine: Real-time syntax checking to highlight invalid connections (e.g., direct data store to data store connection in DFD).

2. Data Flow Diagrams (DFD) — Multi-Level Hierarchy

Data Flow Diagrams visualize the flow of data through an information system. DFDs emphasize data movement, processing, and storage rather than control flow logic.

2.1 Key Components & Notations

Supported Notation Sets:

- Gane & Sarson Notation (Square processes with rounded corners / shaded header bar).
- Yourdon & DeMarco Notation (Circles / bubbles for processes).

Standard Elements:

1. External Entity (Terminator):

- *Visual*: Rectangle or double rectangle.
- *Features*: Represents external people, systems, or organizations providing input or receiving output.

Attribute: `Entity Name`.

2. Process:

- ***Visual*:** Circle or Rounded Rectangle split into sections (Process ID, Process Name, Execution Location/System).
- ***Features*:** Transforms input data flows into output data flows. Must have at least one input and one output.

3. Data Store:

- ***Visual*:** Open-ended rectangle (two parallel lines) or rectangle with a vertical stripe.
- ***Features*:** Represents data at rest (database, file, queue). Attributes: `Store ID` (e.g., D1, D2), `Store Name`.

4. Data Flow:

- ***Visual*:** Directed arrow with data packet label.
- ***Features*:** Represents data in motion. Attributes: `Flow Name`, `Data Structure/Fields`.

2.2 DFD Multi-Level Breakdown & Features

DFDs use hierarchical decomposition to drill down from high-level system overview to fine-grained process logic.

```
graph TD
    Level0["Level 0: Context Diagram (Whole System = 1 Bubble)"]
    Level1["Level 1: System Overview (Major Subsystems D1..Dn, Processes 1.0..N.0)"]
    Level2["Level 2: Child Process Decomposition (Decompose Process 1.0 -> 1.1, 1.2, 1.3)"]
    Level3["Level 3+: Primitive DFD (Detailed Data Transformations & Specifications)"]

    Level0 --> Level1
    Level1 --> Level2
    Level2 --> Level3
```

Level 0 — Context Diagram

- Scope: High-level overview of the entire system as a single process bubble (Process 0).
- Features:
 - Contains exactly one process bubble (Process 0: "Main System").
 - Displays all External Entities surrounding the system.
 - Shows primary incoming data inputs and outgoing data outputs.
 - No Data Stores are shown at Level 0 (they are encapsulated inside the system).

Level 1 — System Overview DFD

- Scope: Decomposes Process 0 into major sub-processes (e.g., Process 1.0, Process 2.0, Process 3.0).
- Features:
 - Introduces Data Stores (D1, D2, D3...).
 - Retains all External Entities from Level 0 to maintain Level Balancing.
 - Shows major data paths between sub-processes, data stores, and external entities.

Level 2 — Sub-Process Decomposition DFD

- Scope: Decomposes a specific Level 1 process (e.g., Process 2.0) into sub-processes (Process 2.1, 2.2, 2.3).
- Features:
 - Shows granular internal logic, intermediate data stores, and temporary data flows.
 - Preserves exact boundary inputs/outputs of the parent Level 1 process.

Level 3+ — Primitive DFD

- Scope: Final level of decomposition where sub-processes cannot be broken down further.
- Features:
 - Accompanied by Process Specifications (P-Specs) written in pseudocode, decision tables, or structured English.

DFD Specific Feature Requirements:

- Child Diagram Drill-Down: Double-clicking a process bubble opens or navigates to its decomposed Child DFD page.

- Strict Level Balancing Checker: Automatically verifies that inputs and outputs of a parent process match the inputs and outputs of its child diagram.
- Syntax Validation Rules:
 - 'L *Error*: Direct flow from External Entity to External Entity.
 - 'L *Error*: Direct flow from External Entity to Data Store (must go through a Process).
 - 'L *Error*: Direct flow from Data Store to Data Store (must go through a Process).
 - 'L *Error*: Process with only inputs ("Black Hole") or only outputs ("Miracle").

3. Use Case Diagrams

Use Case Diagrams describe the functional requirements of a system by showing interactions between actors and use cases.

3.1 Key Components & Notations

1. System Boundary:
 - *Visual*: Large rectangle enclosing use cases, with the system name at the top.
 - *Features*: Separates what is inside the system from external actors.
2. Actor:
 - *Visual*: Stick figure (human actor) or rectangle labeled `<<system>>` (external system).
 - *Features*: Represents roles played by users, external devices, or external software systems interacting with the use case.
3. Use Case:
 - *Visual*: Horizontal oval containing a descriptive verb-noun phrase (e.g., "Authenticate User", "Process Payment").
 - *Features*: Units of custom functionality executed by the system.

3.2 Relationships & Features

```
classDiagram
    class Actor {
        +Primary Actor
        +Secondary Actor
    }
    class UseCase {
        +Goal
        +Preconditions
        +Postconditions
    }
    Actor -- UseCase : Association
    UseCase <.. IncludeUseCase : <<include>>
    UseCase <.. ExtendUseCase : <<extend>>
```

- Association Link: Solid line connecting an Actor to a Use Case.
- Include Relationship (`<<include>>`):
 - *Visual*: Dashed arrow pointing from base Use Case to mandatory sub-use case.
 - *Meaning*: Expresses mandatory common behavior shared across multiple use cases (e.g., `Verify Credentials`).
- Extend Relationship (`<<extend>>`):
 - *Visual*: Dashed arrow pointing from optional extension Use Case to base Use Case, labeled with an optional `Extension Point`.
 - *Meaning*: Expresses conditional or optional functionality (e.g., `Apply Coupon Code`).
- Generalization / Inheritance:
 - *Visual*: Solid line with a hollow triangle arrow pointing to the parent.
 - *Usage*: Actor hierarchy (e.g., `Admin` inherits from `User`) or Use Case hierarchy (e.g., `Pay via Credit Card` inherits from `Process Payment`).

Use Case Diagram Specific Feature Requirements:

- Detailed Scenario Editor: Side panel to define Use Case Metadata (Goal, Primary Actor, Pre-conditions, Post-conditions, Main Success Scenario steps, Alternative Flows).
- Actor Role Matrix: Automatic matrix mapping Actors to Use Cases to ensure no orphan use cases or disconnected actors exist.

4. Entity-Relationship (ER) Diagrams

ER Diagrams model the logical structure of database schemas, detailing entities, attributes, and relationships.

4.1 Key Components & Notations

Supported Styles:

- Chen Notation (Rectangles for entities, Ovals for attributes, Diamonds for relationships).
- Crow's Foot Notation (Rectangles with partitioned sections for PK/FK attributes, lines with crow's foot endpoints for cardinality).

Standard Elements:

1. Entity:
 - Strong Entity: Standard rectangle (independent table).
 - Weak Entity: Double rectangle (depends on parent entity for identification).
2. Attributes (Chen Notation):
 - Simple Attribute: Oval connected to entity.
 - Primary Key Attribute: Oval with underlined text.
 - Multi-valued Attribute: Double oval (e.g., `Phone Numbers`).
 - Derived Attribute: Dashed oval (e.g., `Age` derived from `Date of Birth`).
 - Composite Attribute: Hierarchical branch of ovals (e.g., `Address` -> `Street`, `City`, `Zip`).

4.2 Relationships & Cardinality

In Crow's Foot Notation:

Notation Marker | Meaning

` | ` | One and only one (Mandatory Single)
 ` | o` | Zero or one (Optional Single)
 `} | ` | One or many (Mandatory Multiple)
 `}o` | Zero or many (Optional Multiple)

```
erDiagram
    CUSTOMER ||--o{ ORDER : places
    ORDER ||--|{ ORDER_ITEM : contains
    PRODUCT ||--o{ ORDER_ITEM : includes
```

ER Diagram Specific Feature Requirements:

- Dual View Mode: Toggle between Conceptual (Chen Notation) and Logical/Physical Database Schema (Crow's Foot).
- Automated Data Type Specification: Assign data types (`INT`, `VARCHAR(255)`, `TIMESTAMP`, `BOOLEAN`) and constraints (`NOT NULL`, `UNIQUE`, `AUTO\_INCREMENT`).
- SQL DDL Code Generation: One-click generation of executable SQL scripts (PostgreSQL, MySQL, SQLite, MS SQL Server) directly from the visual ER diagram.
- Reverse Engineering: Import SQL schema files (`.sql`) to automatically generate the visual ER diagram.

5. Activity Diagrams

Activity Diagrams represent the operational step-by-step workflows of components in a system, showing control flow, sequential execution, branching, parallel processing, and swimlanes.

5.1 Key Components & Notations

1. Initial Node (Start): Filled black circle (•).

- 2. Final Activity Node (End): Bullseye circle with filled inner circle (⦿).
- 3. Flow Final Node: Circle with an 'X' (terminates a specific flow path without ending the whole activity).
- 4. Action / Activity State: Rectangle with rounded corners containing the action description.
- 5. Control Flow Line: Directed arrow indicating execution sequence.

5.2 Control Flow Elements & Swimlanes

```
stateDiagram-v2
    [*] --> SubmitOrder
    SubmitOrder --> CheckInventory
    state CheckInventory <<choice>>
    CheckInventory --> PaymentProcess: Item In Stock
    CheckInventory --> RejectOrder: Out of Stock

    state ForkNode <<fork>>
    PaymentProcess --> ForkNode
    ForkNode --> SendEmail
    ForkNode --> PackageItem

    state JoinNode <<join>>
    SendEmail --> JoinNode
    PackageItem --> JoinNode
    JoinNode --> [*]
```

- 1. Decision & Merge Node:
 - ***Visual*:** Diamond symbol (◊).
 - ***Decision*:** 1 incoming flow, multiple outgoing flows with guard conditions `[Condition]`.
 - ***Merge*:** Multiple incoming flows, 1 outgoing flow (recombines conditional paths).
- 2. Fork & Join Bars:
 - ***Visual*:** Heavy solid horizontal or vertical bar.
 - ***Fork*:** 1 incoming flow, multiple outgoing parallel execution threads.
 - ***Join*:** Multiple parallel incoming flows, wait until all complete, then 1 outgoing flow.
- 3. Swimlanes (Partitioning):
 - ***Visual*:** Vertical or horizontal lanes grouping activities by actor, department, or system module (e.g., `Customer`, `Order Service`, `Payment Gateway`).

Activity Diagram Specific Feature Requirements:

- Guard Condition Property Panel: Easily add expression labels to decision branches (e.g., `[payment_approved == true]`).
- Swimlane Drag-and-Drop Organization: Re-order, add, or move action nodes between swimlanes with automatic ownership updating.
- Execution Flow Simulation / Stepper: Interactive step-by-step visual animation highlighting the active token moving through control nodes, decisions, forks, and joins.

6. Diagram Comparison & Feature Summary Matrix

Diagram Type	Primary Purpose	Key Shapes / Elements	Unique / Advanced Feature Requirements
DFD (Data Flow Diagram)	Data movement & transformation modeling	Process, Data Store, External Entity, Data Flow	Hierarchical level drill-down (Level 0, 1, 2), Level balancing checking, Syntax rule validation
Use Case Diagram	Functional requirements & actor interactions	Actor, Use Case, System Boundary, Extend/Include links	Scenario step builder, Actor-Use Case mapping matrix, Extension point markers
ER Diagram	Database schema structure & relationship modeling	Entity (Strong/Weak), Attributes (PK/FK/Multi), Cardinalities	SQL DDL code generation, Crow's foot & Chen notation toggle, Reverse SQL import
Activity Diagram	Workflow & sequential/parallel control flow	Action states, Decisions, Fork/Join bars, Swimlanes	Swimlane auto-partitioning, Guard condition logic builder, Interactive execution flow simulator

7. Recommended Standard Diagram Creation Interface Features

When creating a software editor or interface for these diagrams, include the following UI panels:

1. Left Sidebar — Stencil Palette: Tabs for DFD, Use Case, ER, and Activity primitives.
2. Center Canvas — Interactive Surface: Zoomable grid with real-time snap alignment, dynamic connector handles, and multi-selection box.
3. Right Sidebar — Property & Inspector Panel:
 - Node name, font size, stroke color, fill gradient.
 - Diagram specific metadata (e.g., DFD process P-Specs, ER SQL types, Use Case preconditions).
4. Top Bar — Toolbar & Actions:
 - Level Navigation Breadcrumbs (e.g., `Root > Level 0 > Process 2.0 (Level 1) > Process 2.4 (Level 2)`).
 - Layout auto-arrange buttons, Syntax Validator trigger, SQL / Code exporter, Image PNG/SVG downloader.

